

Lab: 0110

Shift Registers & Counters

From Flip-Flops to Sequencing Circuits

CSA222: Modern Computer Architecture

Lab Roadmap

Two theory blocks, following the same order and depth as the Neso Academy Digital Electronics playlist — then six build exercises.

A

Registers & Shift Registers

Intro to registers → data formats → SISO → SIPO → PISO → PIPO → bidirectional → universal shift register.

B

Counters

Asynchronous (ripple) up/down → synchronous design method → 2/3/4-bit up/down → Ring Counter → Johnson Counter.

C

Verilog Exercises

6 build questions: two registers, two shift registers, two synchronous counters — problem + solution each.

Registers & Shift Registers

Storage elements built from flip-flops — how data enters, moves, and leaves.

1

What is a Register?

A group of flip-flops sharing one clock.

2

Data Formats

Serial vs. parallel in, serial vs. parallel out.

3

4 Basic Modes

SISO, SIPO, PISO, PIPO.

4

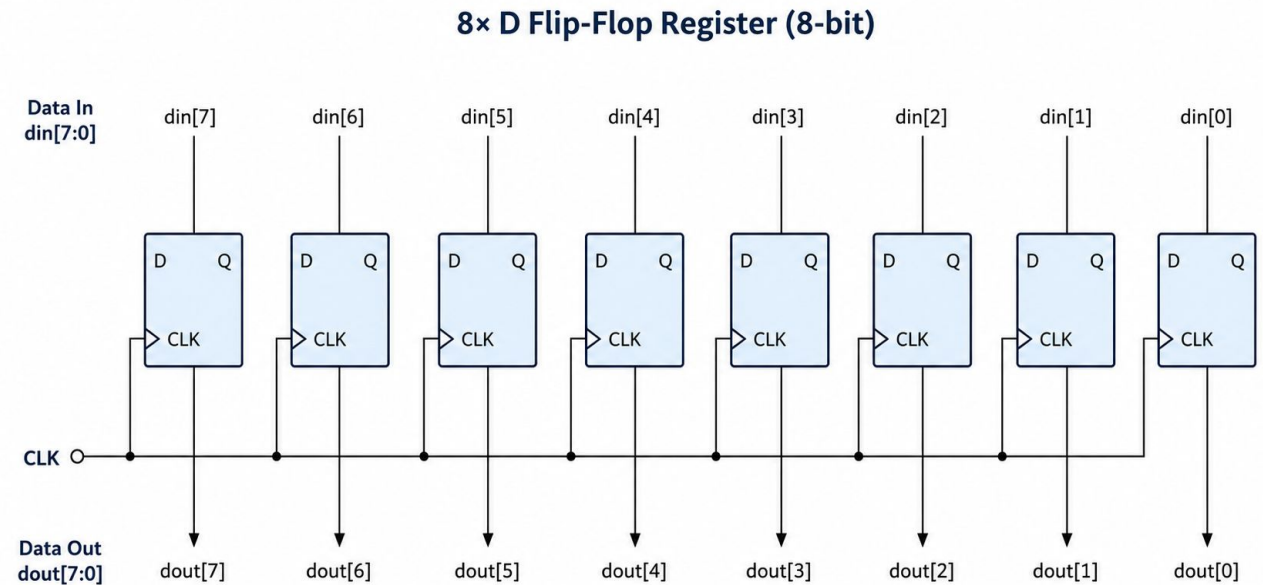
Advanced Modes

Bidirectional & Universal shift registers.

What Is a Register?

A register is a group of flip-flops, each storing one bit, all sharing a common clock.

- ◆ An n-bit register = n flip-flops in parallel, clocked together, so an n-bit word updates atomically
- ◆ Registers are the fastest storage a digital system has — no addressing or bus arbitration needed, just a clock edge
- ◆ Two control lines are common on a practical register: LOAD (accept new data) and CLEAR (force all bits to 0)
- ◆ Registers differ from plain flip-flop banks only in how data is organised: as a stored word with defined I/O behaviour
- ◆ Building block for: CPU register files, pipeline stage latches, the Program Counter, and every shift register on the next slides



Are registers made of flip-flops? Yes — always. Everything from here builds on this one fact.

Data Formats and Classification of Registers

Every register is classified by how data enters and how it is read back out.

- Serial format: bits arrive/leave one at a time, on successive clock edges, over a single wire
- Parallel format: all bits arrive/leave together, in one clock edge, over n wires

Mode	Input	Output	Typical Use
SISO	Serial	Serial	Pure delay line, data buffering
SIPO	Serial	Parallel	Serial-to-parallel conversion (e.g. UART receive)
PISO	Parallel	Serial	Parallel-to-serial conversion (e.g. UART transmit)
PIPO	Parallel	Parallel	Ordinary buffer register with load enable

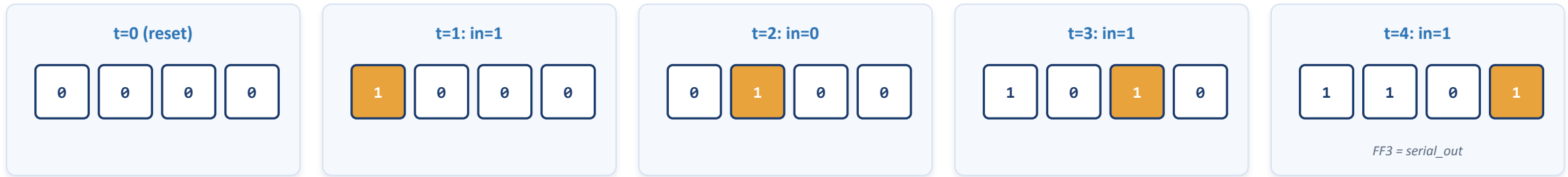
Bidirectional and Universal registers (slides A7–A8) extend this table with a direction/mode control.

SISO: Serial-In, Serial-Out

Each flip-flop's Q feeds the next flip-flop's D. Data enters one bit per clock and exits one bit per clock, n cycles later.

✦ Shift-right convention: bit walks FF0 -> FF1 -> FF2 -> FF3, one hop per rising edge.

FILMSTRIP — shifting in 1,0,1,1 (serial_in shown at left of each frame)

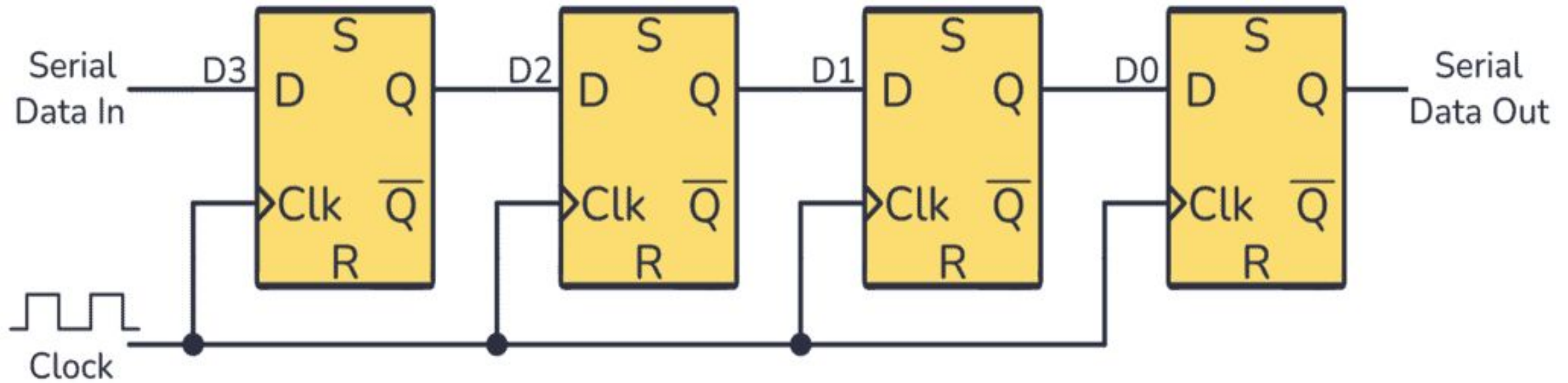


```
always @(posedge clk) begin
    q <= {q[2:0], serial_in}; // shift right: q3<-q2<-q1<-q0<-serial_in
end
assign serial_out = q[3];    // read one bit out, one bit at a time
```

After 4 clocks the input word has fully entered — but you can only read it 1 bit at a time out of serial_out.

SISO: Serial-In, Serial-Out

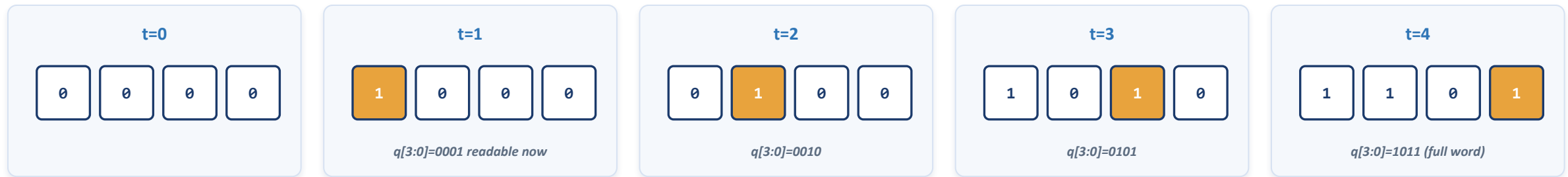
Each flip-flop's Q feeds the next flip-flop's D. Data enters one bit per clock and exits one bit per clock, n cycles later.



SIPO: Serial-In, Parallel-Out

Same shifting chain as SISO — the difference is purely in how you read the output: tap every flip-flop's Q at once.

FILMSTRIP — same 1,0,1,1 input, but every Q is available immediately (parallel taps)

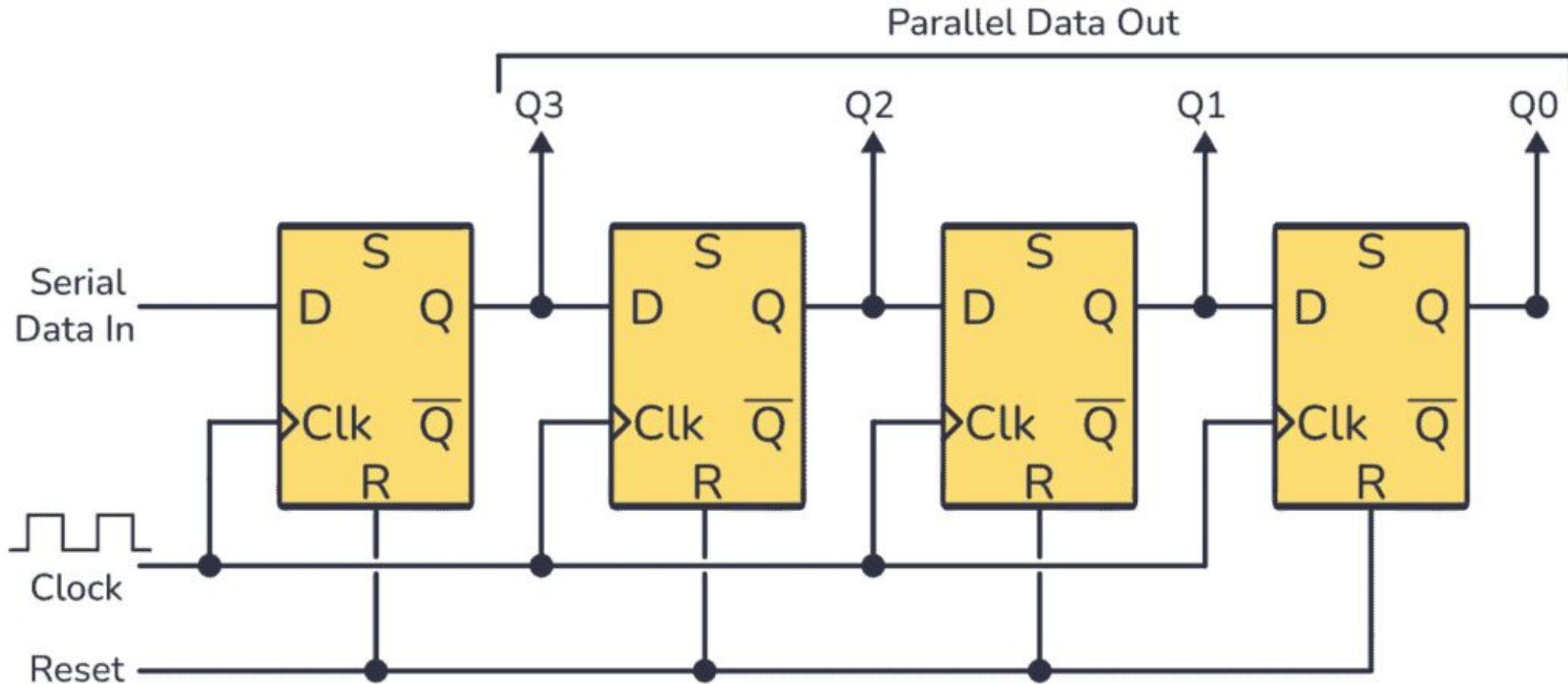


- ✦ Used to reassemble a serial stream (UART RX, SPI MISO) back into a parallel word for the rest of the system
- ✦ No extra hardware vs SISO — it is the SAME circuit; the four Q outputs were always there
- ✦ This is exactly the sipo_reg circuit you'll build in Exercise Q3

```
module sipo_reg(
    input clk, sin,
    output reg [3:0] q);
    always @(posedge clk)
        q <= {q[2:0], sin};
endmodule // q IS the parallel out
```

SIPO: Serial-In, Parallel-Out

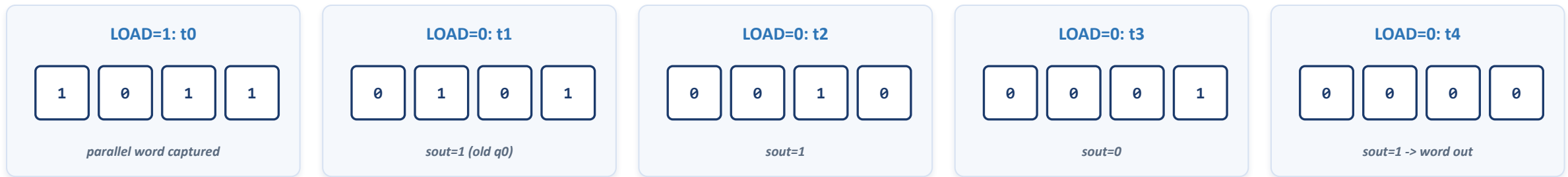
Same shifting chain as SISO — the difference is purely in how you read the output: tap every flip-flop's Q at once.



PISO: Parallel-In, Serial-Out

Load an entire word in one clock edge, then shift it out one bit per clock — needs a 2:1 mux in front of every D input.

FILMSTRIP — load 1011 in one edge, then shift out right-to-left

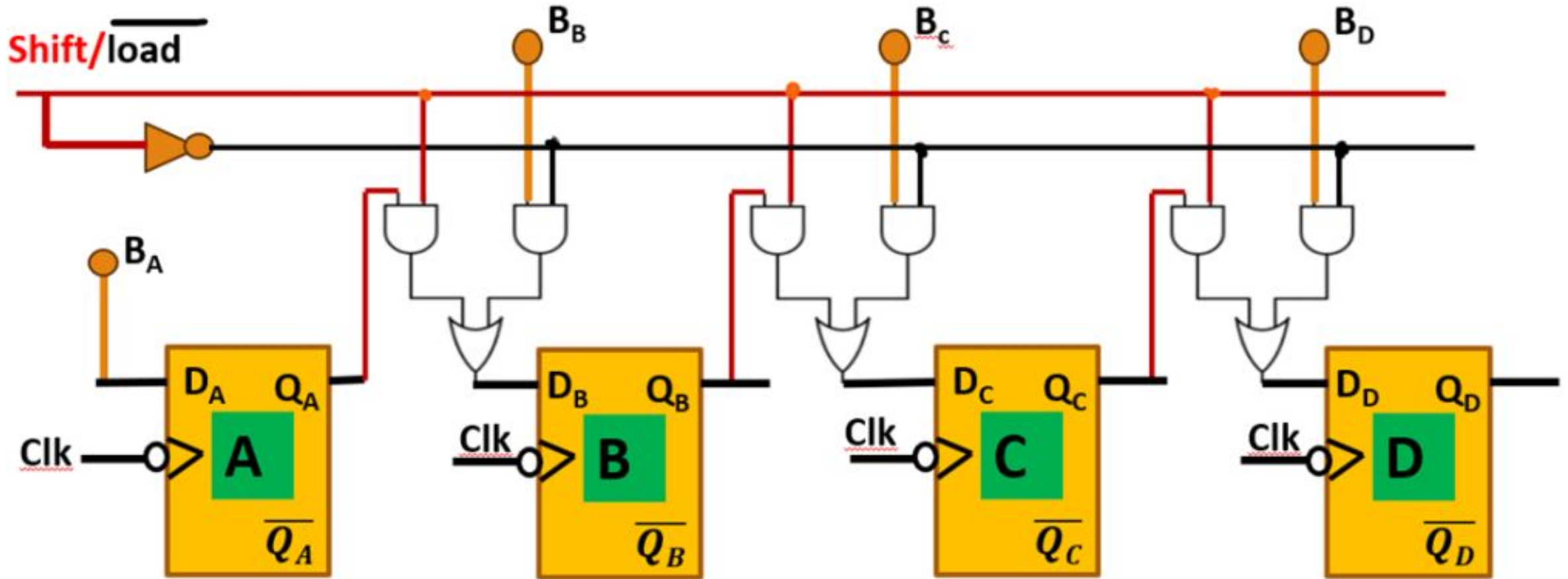


- ✦ Each flip-flop's D input is a MUX: LOAD=1 selects the parallel data bit, LOAD=0 selects the neighbour's Q (shift)
- ✦ This is the mirror image of PIPO's load-enable idea, combined with SISO's shifting idea
- ✦ Classic use: convert a wide parallel bus into a narrow serial line (e.g. driving an SPI shift-out chip)

```
always @(posedge clk) begin
    if (load) q <= d;           // parallel load
    else      q <= {1'b0, q[3:1]}; // shift right
end
assign sout = q[0];
```

PISO: Parallel-In, Serial-Out

Load an entire word in one clock edge, then shift it out one bit per clock — needs a 2:1 mux in front of every D input.



PIPO: Parallel-In, Parallel-Out

No shifting at all — every bit loads and reads together. This is simply a load-enable register (Slide A1) wearing the 'shift register' family name.

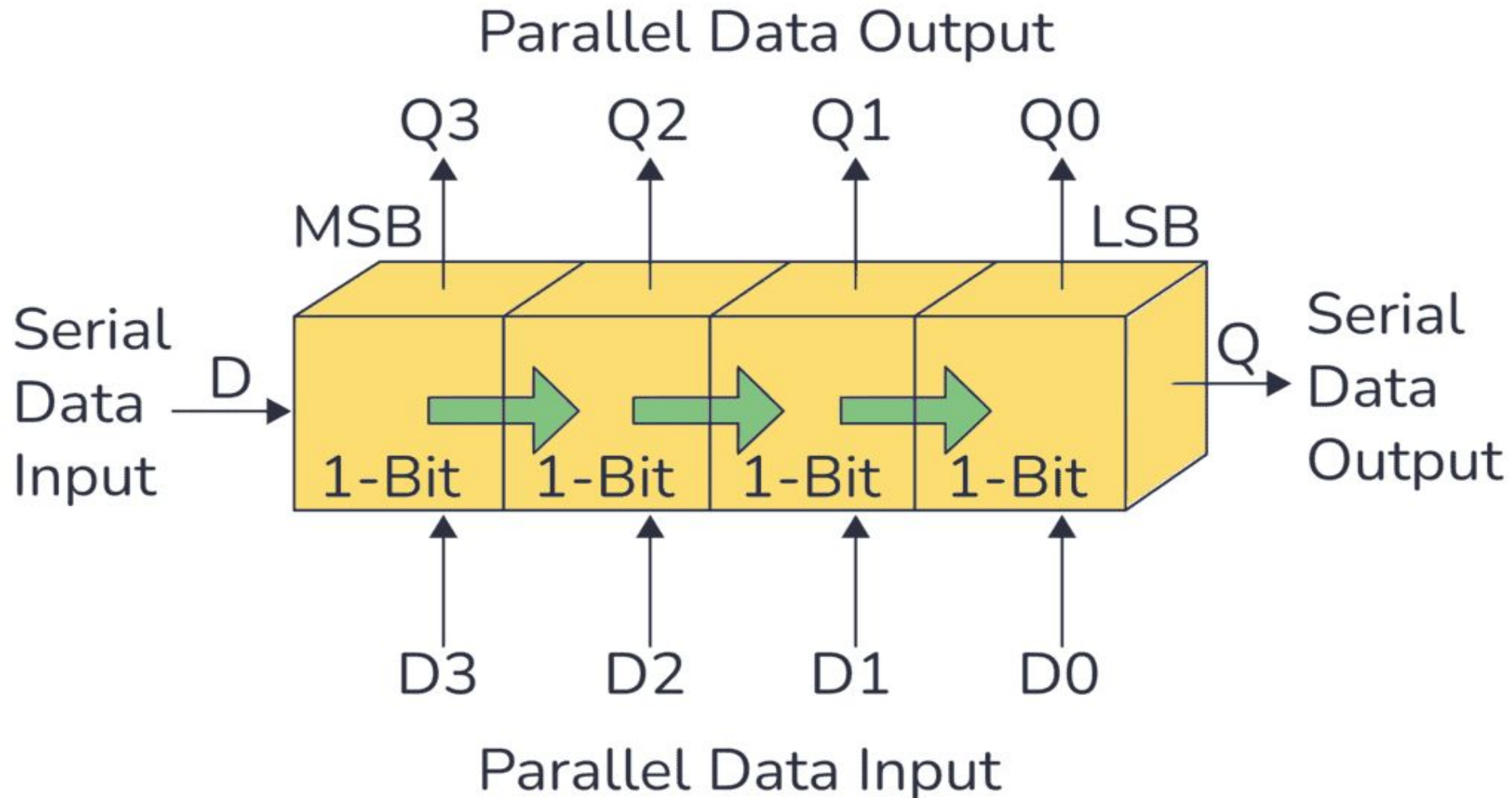
- ✦ On a clock edge with LOAD=1, all n bits update simultaneously from d[n-1:0]
- ✦ With LOAD=0, the register holds — no shifting, no serial ports at all
- ✦ It's grouped with the shift-register family because it shares the same D-flip-flop-row hardware, just without the neighbour-to-neighbour wiring
- ✦ This is the exact behaviour tested in Exercise Q1 and Q2 (8-bit register, with and without synchronous reset)

```
always @(posedge clk)
  if (load)
    q <= d;
  // else: hold
```

Compare to A3 (SISO): same register skeleton, different D-input wiring — neighbour's Q vs. an external bus.

PIPO: Parallel-In, Parallel-Out

No shifting at all — every bit loads and reads together. This is simply a load-enable register (Slide A1) wearing the 'shift register' family name.



Counters

A counter is a register that always overwrites itself with the next count value — no external data input needed.

1

Asynchronous

Ripple counters — up, down, up/down.

2

Synchronous

Excitation-table design method; 2/3/4-bit up/down.

3

Ring Counter

One '1' circulates through n flip-flops.

4

Johnson Counter

Complemented feedback — $2n$ unique states.

Asynchronous (Ripple) Up Counter — 2-Bit

Each flip-flop toggles ($T=1$) using the PREVIOUS flip-flop's Q as its own clock — the count ripples through, stage by stage.

- ✦ Only FF0 sees the real clock; FF1 is clocked by FF0's Q, FF2 by FF1's Q, and so on
- ✦ Every stage is wired $T=1$, so it simply toggles every time it is clocked
- ✦ This reproduces the same divide-by-2 chain from Lecture 6 — reading it as bits IS counting
- ✦ Simple to build (no extra logic gates) but has ripple delay — real count is only valid after all stages settle

FILMSTRIP — 2-bit ripple up counter (Q1 Q0), 4 clock edges



Q0 toggles every clock; Q1 toggles only when Q0 falls 1→0 — exactly like a divide-by-2 chain.

3-Bit & 4-Bit Ripple Counters, and Their Speed Limit

The 2-bit pattern extends directly: add one more T flip-flop, clocked by the previous stage's Q, per extra bit.

- ✦ 3-bit: FF0 → FF1 → FF2, counts 000 through 111, then wraps
- ✦ 4-bit: one more stage, counts 0000 through 1111 (MOD-16)
- ✦ In general, n ripple stages count MOD-2ⁿ and divide the input clock by 2ⁿ

After edge	Q3 Q2 Q1 Q0	Value
before	0 1 1 1	7 (correct)
+1 tpd	0 1 1 0	6 (wrong, transient)
+2 tpd	0 1 0 0	4 (wrong, transient)
+4 tpd	1 0 0 0	8 (correct at last)

THE SPEED LIMIT

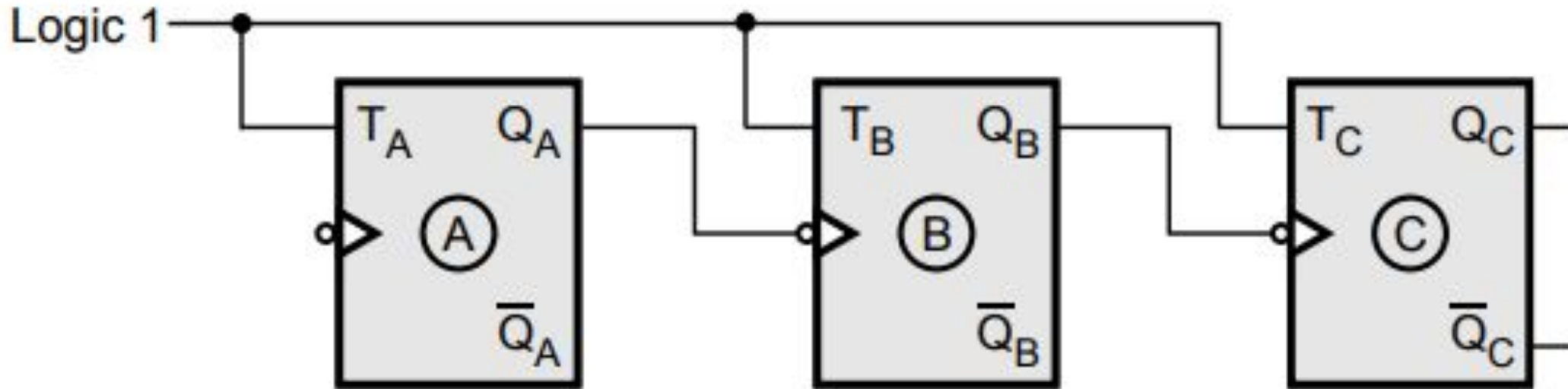
$$t_{\text{settle}} = n \times t_{\text{pd}}$$

$$f_{\text{max}} = 1 / (n \times t_{\text{pd}})$$

Any circuit reading the count while it ripples — a comparator, a memory address bus — briefly sees the wrong value. At $t_{\text{pd}} = 10 \text{ ns}$, a 4-bit ripple counter tops out near 25 MHz; a modern CPU clock is ~3 GHz. This is exactly why synchronous counters (next slides) exist.

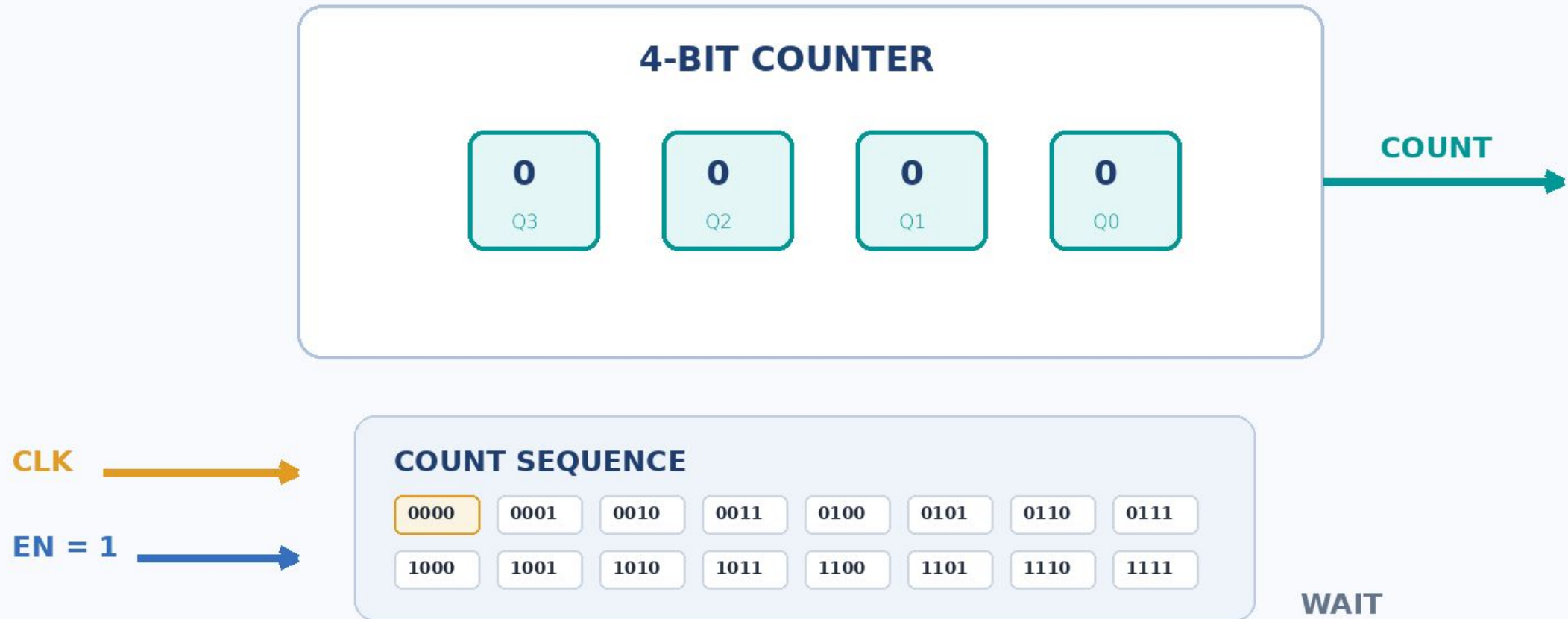
3-Bit & 4-Bit Ripple Counter, and Their Speed Limit

The 2-bit pattern extends directly: add one more T flip-flop, clocked by the previous stage's Q, per extra bit.



4-BIT SYNCHRONOUS COUNTER

Binary count advances by 1 at each rising clock edge



$Q \leftarrow Q + 1 \pmod{16} \quad 0000 \rightarrow 0001 \rightarrow 0010 \rightarrow \dots \rightarrow 1111 \rightarrow 0000$

How to Design a Synchronous Counter

Every flip-flop shares the real clock; combinational logic in front decides each flip-flop's next input.

1

Write the state (count) sequence you want, e.g. 00 -> 01 -> 10 -> 11 -> 00

2

Build a present-state / next-state table from that sequence

3

For each flip-flop, use its excitation table (Lecture 6) to find what input makes that $Q \rightarrow Q'$ transition

4

K-map each flip-flop's input as a function of the present-state bits

5

Wire the resulting gates to each flip-flop's input; all flip-flops still share one clock

This is the same 6-step FSM design flow from Lecture 6 — a counter is just an FSM whose states happen to be binary numbers.

Worked Example: 2-Bit Synchronous Up Counter

Q1 Q0	Next Q1 Q0	T1	T0
0 0	0 1	0	1
0 1	1 0	1	1
1 0	1 1	0	1
1 1	0 0	1	1

- ✦ $T0 = 1$ always $\rightarrow$ Q0 toggles every clock (same as the ripple LSB)
- ✦ $T1 = Q0 \rightarrow$ Q1 only toggles when Q0 was 1 the previous edge
- ✦ Both flip-flops now share the SAME real clock — no ripple delay

```
module up2 (  
    input clk, rst,  
    output reg [1:0] q  
);  
    wire T0 = 1'b1;  
    wire T1 = q[0];  
    always @(posedge clk) begin  
        if (rst) q <= 2'b00;  
        else begin  
            q[0] <= q[0] ^ T0;  
            q[1] <= q[1] ^ T1;  
        end  
    end  
endmodule
```

3-Bit Synchronous Up Counter — the General Rule

Extending the 2-bit result reveals the pattern used for any width.

THE RULE

$$T_0 = 1$$

$$T_1 = Q_0$$

$$T_2 = Q_0 \cdot Q_1$$

A bit toggles exactly when every bit below it is 1 — identical to the ripple-counter rule from Lecture 6, just computed combinatorially instead of chained through real hardware clocking.

```
module up3 (  
    input clk, rst,  
    output reg [2:0] q  
);  
always @(posedge clk) begin  
    if (rst) q <= 3'b000;  
    else      q <= q + 3'b001;  
    // synthesiser derives  
    // T0/T1/T2 automatically  
    // from the '+1' -- this  
    // is why real designs  
    // write q+1, not manual  
    // T-equations.  
end  
endmodule
```

3-Bit & 4-Bit Synchronous Up/Down Counter

Add one control input; the next-state adder now adds +1 or -1 depending on it — everything else is unchanged.

- ✦ `up_down = 1` -> `q <= q + 1` (counts up, wraps MAX -> 0)
- ✦ `up_down = 0` -> `q <= q - 1` (counts down, wraps 0 -> MAX)
- ✦ Still fully synchronous: one clock, one clean transition per edge
- ✦ This is exactly the difference between Exercise Q5 (up) and Q6 (down) later in this deck

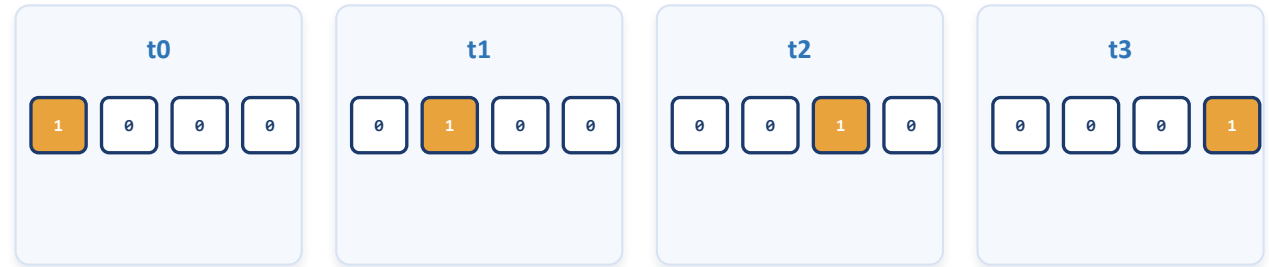
```
module updown4 (  
    input clk, rst, up_down,  
    output reg [3:0] q  
);  
    always @(posedge clk) begin  
        if (rst) q <= 4'b0000;  
        else if (up_down)  
            q <= q + 4'b0001;  
        else  
            q <= q - 4'b0001;  
        end  
    endmodule
```

Ring Counter

A shift register (SISO-style) with its output fed back to its own input — a single 1 circulates forever.

- ✦ **n** flip-flops, connected exactly like a shift register, but Q(last) feeds back to D(first)
- ✦ Preset exactly one flip-flop to 1 and clear the rest — that single 1 then walks around the ring, one position per clock
- ✦ **n** flip-flops give exactly **n** unique output states (one-hot) — no decoding logic needed to know which state you're in
- ✦ Used for simple round-robin sequencing: LED chasers, one-hot state machines, simple timing generators

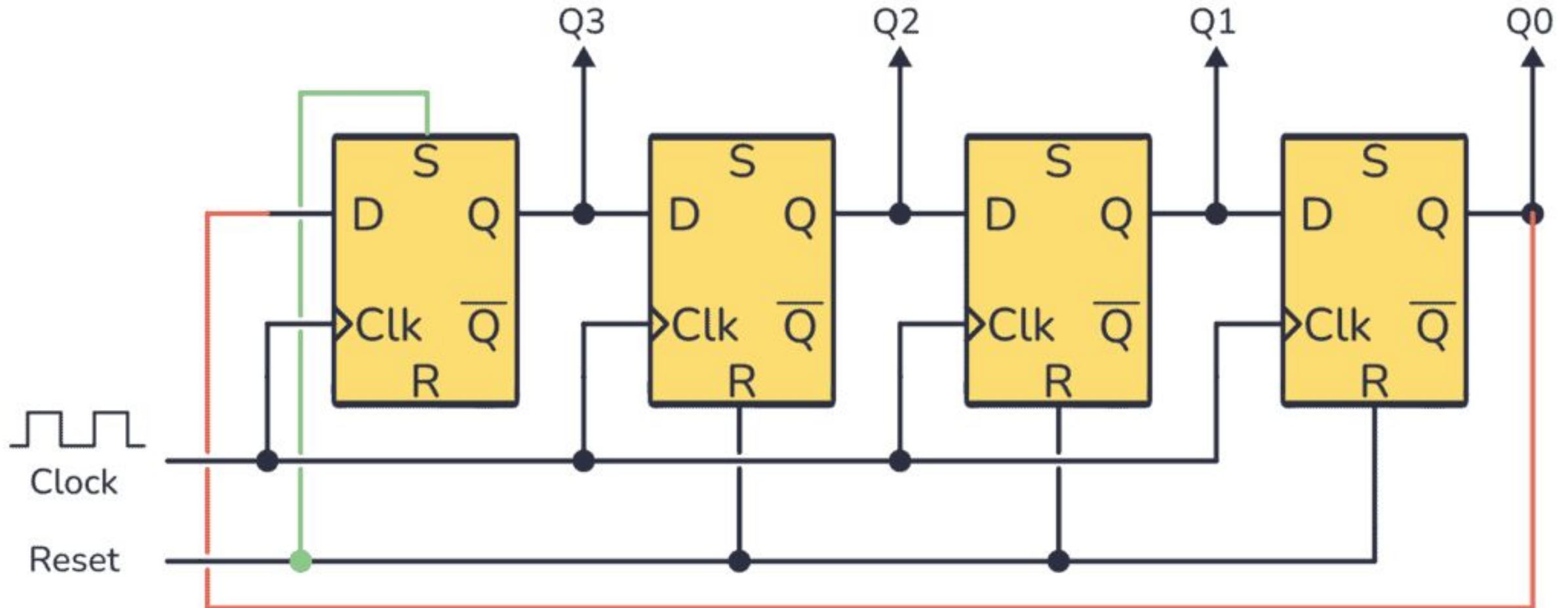
FILMSTRIP — 4-bit ring counter, one 1 circulating



```
always @(posedge clk)
  if (rst) q <= 4'b0001; // preset one 1
  else    q <= {q[2:0], q[3]}; // feedback
```

Ring Counter

A shift register (SISO-style) with its output fed back to its own input — a single 1 circulates forever.



Ripple vs Synchronous vs Ring vs Johnson

Counter	Clocking	States (n stages)	Extra Logic	Best For
Ripple (async)	Chained	2^n	None	Cheap, low-speed dividers
Synchronous	Shared	2^n	Next-state gates/adder	Any speed-critical count (PC, timers)
Ring	Shared	n	None (just feedback)	One-hot sequencing, LED chasers
Johnson	Shared	2n	None (complemented feedback)	More states than Ring, still glitch-free

All four are the same skeleton — a row of flip-flops with a shared clock — differing only in what feeds each D input.

Verilog Build Exercises

Six designs, drawn directly from the register / shift-register / counter theory just covered.

1-2

Registers

Load-enable, then add synchronous reset.

3-4

Shift Registers

SIPO, then PISO.

5-6

Counters

4-bit synchronous up, then down.

Q1

8-bit Register with Load Enable

PROBLEM

- ✦ Write reg8_load: inputs clk, load, d[7:0]; output q[7:0].
- ✦ On a rising edge: if load=1, capture d into q.
- ✦ If load=0, q holds its previous value (not X, not 0).
- ✦ No reset pin required for this exercise.

SOLUTION

```
module register_8bit (  
    input      clk,  
    input      ld,  
    input  [7:0] d,  
    output reg [7:0] q  
);  
    always @(posedge clk) begin  
        if (ld)  
            q <= d;  
    end  
endmodule
```

Q2

8-bit Register with Synchronous Reset

PROBLEM

- ◆ Extend Q1: add a synchronous rst input.
- ◆ On a rising edge, if rst=1, q clears to 8'b0 — regardless of load.
- ◆ Priority order: rst beats load beats hold.
- ◆ rst only takes effect on a clock edge (NOT asynchronous).

SOLUTION

```
module register_8bit_rst (  
    input        clk,  
    input        rst,  
    input        ld,  
    input  [7:0] d,  
    output reg [7:0] q  
);  
    always @(posedge clk) begin  
        if (rst)  
            q <= 8'b00000000;  
        else if (ld)  
            q <= d;  
    end  
endmodule
```

Q3

SIPO Shift Register

PROBLEM

- ✦ Write `sipo_reg`: a 4-bit Serial-In, Parallel-Out shift register.
- ✦ Inputs: `clk`, `sin` (1 bit). Output: `q[3:0]` (parallel).
- ✦ Each rising edge: shift `q` left by one, load `sin` into bit 0.
- ✦ After 4 clocks, `q[3:0]` holds the last 4 bits received.

SOLUTION

```
module sipo (  
    input      clk,  
    input      rst,  
    input      d_in,  
    output reg [3:0] q  
);  
    always @(posedge clk) begin  
        if (rst)  
            q <= 4'b0000;  
        else begin  
            q[3] <= q[2];  
            q[2] <= q[1];  
            q[1] <= q[0];  
            q[0] <= d_in;  
        end  
    end  
endmodule
```

Q4

PISO Shift Register

PROBLEM

- ✦ Write `piso_reg`: a 4-bit Parallel-In, Serial-Out shift register.
- ✦ Inputs: `clk`, `load`, `d[3:0]`. Output: `sout` (1 bit).
- ✦ If `load=1` on a clock edge, capture `d` (parallel load).
- ✦ If `load=0`, shift right each edge; output the LSB on `sout`.

SOLUTION

```
module piso (  
    input        clk,  
    input        rst,  
    input        sh_ld,  
    input [3:0] d,  
    output reg [3:0] q,  
    output       d_out  
);  
    assign d_out = q[0];  
  
    always @(posedge clk) begin  
        if (rst)  
            q <= 4'b0000;  
        else if (!sh_ld)  
            q <= d;  
        else begin  
            q[0] <= q[1];  
            q[1] <= q[2];  
            q[2] <= q[3];  
            q[3] <= 1'b0;  
        end  
    end  
endmodule
```

Q5

4-bit Synchronous Up Counter

PROBLEM

- ✦ Write up_counter4: a 4-bit synchronous up counter.
- ✦ Inputs: clk, rst, en. Output: q[3:0].
- ✦ On a rising edge: if rst=1, q clears to 0000.
- ✦ Else if en=1, q increments by 1; else, q holds.
- ✦ Must wrap 1111 -> 0000 with no special-case code.

SOLUTION

[illegible]

Q6

4-bit Synchronous Down Counter

PROBLEM

- ✦ Write `down_counter4`: a 4-bit synchronous down counter.
- ✦ Inputs: `clk`, `rst`, `en`. Output: `q[3:0]`.
- ✦ On a rising edge: if `rst=1`, `q` loads to 1111 (its MAX, not 0).
- ✦ Else if `en=1`, `q` decrements by 1; else, `q` holds.
- ✦ Must wrap 0000 -> 1111 with no special-case code.

SOLUTION

```
module down_counter4 (  
    input clk, rst, en,  
    output reg [3:0] q  
);  
    always @(posedge clk) begin  
        if (rst)  
            q <= 4'b1111;    // reset to  
                             // MAX, so it  
                             // has room  
                             // to count  
                             // down  
        else if (en)  
            q <= q - 4'b0001; // wraps  
                             // 0000-1  
                             // =1111  
        end  
    endmodule
```

Key Takeaways

Two theory blocks, one skeleton each.

01

A register is n flip-flops sharing a clock; shift registers just wire neighbour-to-neighbour (SISO/SIPO/PISO/PIPO), and can add direction (bidirectional) or full mode-select (universal).

02

A ripple counter chains flip-flop clocks together — cheap, but the count is briefly wrong while it settles (ripple delay).

03

A synchronous counter shares one real clock; combinational logic computes each flip-flop's next input, so the whole count is valid the instant after the edge.

04

Ring and Johnson counters are shift registers with feedback — n states (Ring) or $2n$ states (Johnson), no decoding logic needed.

05

Every Verilog exercise in this lab is one of these skeletons: an if/else inside always @(posedge clk) — only the next-state expression changes.



**Thanks
for
watching!**